

DD: SUB-WF-UPGRADE-01-FLOW-WIK — KE WIK

Upgrade BPMN

Developer implementation guide: DD_SUB-WF-UPGRADE-01-FLOW-WIK-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow DD_SUB-WF-UPGRADE-01-FLOW-WIK-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0 Satellite of

DD_SUB-WF-UPGRADE-01-Subscription_Upgrade (parent). Defines the KE WIK reference BPMN for subscription upgrade. BPMN process key: sub-upgrade-wik. Composes 13 workers from the framework's WORKERS catalog, 2 message catches, 1 Drools package. Pay-first gate active for KE v1.0 (proration delta collected before commit).

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-UPGRADE-01-FLOW-WIK-v1.0.md
Version	v1.0
DD type	operator BPMN flow
BPMN/process keys	sub-upgrade-wik
Drools packages	rules.subscription.upgrade.wik

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /subscriptions/sub_01HZ_WANJIKU/pending-upgrade/subop_01HZ_WANJIKU_UPGRADE_EOC
- POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/upgrade
- POST /api/subscriptions/sub_01HZ_OTIENO_FIBER/upgrade

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- bil01-charge-pre-authorize
- bil01-confirm-charge
- bil01-emit-intent
- bil01-revert-pending-charge
- compute-equipment-delta
- drools-decide
- ful-call-modify
- fulfillment-completed
- fulfillment-failed
- kafka-emit
- manual-recovery-decision
- notification-send
- osr-equipment-bind
- osr-equipment-unbind
- payment-confirmed
- payment-timeout
- resolve-cycle-anchor-policy
- resolve-effective-timing
- sub-lm-commit-state
- sub-lm-compute-cycle-window
- sub-lm-put-master-fields
- sub-lm-put-pending-status
- sub-lm-revert-to-prior
- sub-upgrade-wik
- subscription-operation-finalize
- subscription-workflow-rules
- validate-homepass-compatibility
- validate-preconditions
- validate-target-package

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- SubscriptionUpgradeCancelled
- SubscriptionUpgradeRejected
- SubscriptionUpgradeScheduled
- SubscriptionUpgraded

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.0

Satellite of **DD_SUB-WF-UPGRADE-01-Subscription_Upgrade** (parent). Defines the KE WIK reference BPMN for subscription upgrade. BPMN process key: sub-upgrade-wik. Composes 13 workers from the framework's WORKERS catalog, 2 message catches, 1 Drools package. Pay-first gate active for KE v1.0 (proration delta collected before commit).

1. What this flow is

KE WIK's BPMN composes the framework workers to upgrade a subscription. Trigger endpoint accepts the request (from customer portal, CC, or BO admin), persists `subscription_operation` row, starts process instance of

sub-upgrade-wik. The BPMN orchestrates: Drools validation (preconditions + target-package + equipment-delta) → pending state flip → compute cycle window → billing intent (proration) → pay-first gate (delta > 0) → FUL-03 MODIFY_SERVICE → equipment-bind step (conditional on equipment-requirement delta) → master fields update → commit state ACTIVE → emit event → notify customer → finalize.

Workers composed (13)

For non-IMMEDIATE upgrades, the BPMN inserts a **Camunda intermediate timer event** before the VALIDATING state — no extra worker needed; the timer is a BPMN-native construct that fires automatically when its `timeDate` is reached. During the wait, `subscription_operation.current_state = SCHEDULED` and the subscription stays on the source Package.

#	Worker topic	Used for
1	drools-decide	Validation: subscription ACTIVE, no in-flight ops, role-trigger, target Package ACTIVE + currency + price, HomePass compatibility, equipment-requirement-delta computation
2	sub-lm-put-pending-status	Flip subscription to PENDING_UPGRADE; record <code>current_transition_type=UPGRADE</code> , <code>current_transition_reason_code=<upgradeTrigger></code>
3	sub-lm-compute-cycle-window	Compute cycle window for billing intent context (cycle days remaining drives proration)
4	bil01-emit-intent	UPGRADE_INTENT — billing computes proration delta + any upgrade BillableEvents
5	bil01-charge-pre-authorize	Conditional — only if <code>pay_first_required = true AND delta > 0</code> ; pay-first gate
6	bil01-confirm-charge	Conditional — confirm payment after <code>payment-confirmed</code>
7	ful-call-modify	FUL-03 with <code>intent=MODIFY_SERVICE</code> . FUL-03 owns the per-service-class fan-out: diff source vs target Package compositions, dispatch <code>modifyService/activate/deactivate</code> to provisioners using HomePass <code>service_management_endpoints</code> for node + port resolution
8	osr-equipment-bind	Conditional — only if equipment-requirement delta requires new bindings
9	osr-equipment-unbind	Conditional — only if <code>unbindObsoleteEquipment=true AND target Package no longer requires service classes</code>
10	sub-lm-put-master-fields	Write <code>package_ref</code> , <code>package_version_id</code> , <code>previous_package_ref</code> , <code>previous_package_version_id</code> , <code>last_status_changed_at</code>
11	sub-lm-commit-state	Flip subscription back to ACTIVE with new Package
12	kafka-emit	Emit <code>SubscriptionUpgraded</code> via outbox
13	notification-send	NOT-01 customer notification (reliability-isolated; only if operator config enables)
—	subscription-operation-finalize	Set <code>subscription_operation.final_state=COMPLETED</code>

Message catches

Type	Name	Used when
Message catch	<code>payment-confirmed</code> (correlated on <code>billableEventId</code>)	Only when <code>bil01-charge-pre-authorize</code> was called; boundary timer 7 days then <code>payment-timeout</code>
Message catch	<code>fulfillment-completed</code> / <code>fulfillment-failed</code> (correlated on <code>fulfillmentCorrelationId</code>)	After <code>ful-call-modify</code> ; BPMN waits for FUL-03

Drools package

`rules.subscription.upgrade.wik` — declared decisions:

Decision	Output	Used at
validate-preconditions	eligible: bool, eligibilityReasonCode?, sourcePackageSnapshot: object	Beginning — subscription ACTIVE, no in-flight, role-trigger match
validate-target-package	targetValid: bool, targetReasonCode?, targetPackageSnapshot: object	Target ACTIVE + currency match + price ≥ source + ≠ source
validate-homepass-compatibility	homePassCompatible: bool, homePassReasonCode?, missingServiceManagementKeys: array, unprovisionedPorts: array	HomePass service_management_endpoints covers target's required service families (by serviceManagementKey); resolved ports are non-null; HomePass status_code is_active
compute-equipment-delta	equipmentBindRequired: bool, unfulfilledServiceClasses: array, equipmentUnbindList: array	Diff source vs target compositions + existing OSR bindings; determines whether bind/unbind steps run
resolve-cycle-anchor-policy	resolvedCycleAnchorPolicy: str, policyValid: bool, policyReasonCode?	Resolves to request value if supplied, else operator-config default; validates enum
resolve-effective-timing	resolvedEffectiveTiming: str, scheduledCommitAt: timestamp?, timingValid: bool, timingReasonCode?	Resolves to request value if supplied, else operator-config default; validates scheduledAt bounds; computes scheduledCommitAt (current cycle end for END_OF_CURRENT_CYCLE, supplied value for SCHEDULED_AT, null for IMMEDIATE); rejects redundant END_OF_CURRENT_CYCLE + RESET combination

2. BPMN structure

Process key: sub-upgrade-wik. Deployed as sub-upgrade-wik.bpmn in the Camunda engine.

2.1 Outline

```

START (event)
  ■ Process variables from trigger endpoint:
  ■ operationId, subscriptionId, customerId, operatorCode, packageRef, homepassId,
  ■ upgradeTrigger, targetPackageRef, cycleAnchorPolicy?, effectiveTiming?, scheduledAt?,
  ■ equipmentBindings, unbindObsoleteEquipment,
  ■ notes, initiatingActorUserId, initiatingActorRole, initiatingWorkflowKind,
  ■ initiatingWorkflowRef, idempotencyKey
  ▼
[STATE: VALIDATING_REQUEST]
Service Task: drools-decide
  inputs: kjarPackage="rules.subscription.upgrade.wik",
         factsJson={subscription state, source Package snapshot, target Package snapshot,
                    target package_version snapshot, HomePass service_management_endpoints + status_code,
                    existing OSR bindings, equipmentBindings request,
                    cycleAnchorPolicy request, effectiveTiming + scheduledAt request,
                    role, operator config},
         requestedDecisions=["validate-preconditions", "validate-target-package",
                             "validate-homepass-compatibility", "compute-equipment-delta",
                             "resolve-cycle-anchor-policy", "resolve-effective-timing"]
  outputs: eligible, eligibilityReasonCode, sourcePackageSnapshot, targetValid,
          targetPackageSnapshot, homePassCompatible,
          homePassReasonCode, missingServiceManagementKeys, unprovisionedPorts,
          equipmentBindRequired, unfulfilledServiceClasses, equipmentUnbindList,
          resolvedCycleAnchorPolicy, policyValid, policyReasonCode,
          resolvedEffectiveTiming, scheduledCommitAt, timingValid, timingReasonCode
  ■
  ▼ XOR: all checks pass?
  ■ NO → REJECT subflow (emit SubscriptionUpgradeRejected + finalize FAILED)
  ■ YES ↓
  ■
  ▼ XOR: resolvedEffectiveTiming == "IMMEDIATE"?
  ■ YES ↓ (go straight to commit sequence)
  ■ NO ↓
  ■ [STATE: SCHEDULED]
  ■ Service Task: kafka-emit
  ■ inputs: topic="sophix.subscription.subscription.upgradescheduled",
  ■ payloadJson=<SubscriptionUpgradeScheduled per parent $6>
  ■ ▼
  ■ Service Task: notification-send (reliability-isolated)
  ■ inputs: notificationEventKind="SUBSCRIPTION_UPGRADE_SCHEDULED",
  ■ eventContext={sourcePackageRef, targetPackageRef, scheduledCommitAt,
  ■ resolvedEffectiveTiming, resolvedCycleAnchorPolicy}
  ■ ▼

```

```

■ Intermediate Catch Event: timer
■   timeDate: ${scheduledCommitAt}
■   Boundary listener: external-cancel-requested (signal from cancel REST endpoint
■                     OR from another state-affecting operation per R-EF-5)
■   → on cancel: emit SubscriptionUpgradeCancelled + finalize CANCELLED
■   ▼ timer fires
■
▼
[STATE: VALIDATING]
Service Task: drools-decide
  inputs: kjarPackage="rules.subscription.upgrade.wik",
         factsJson=<re-loaded current snapshots – subscription, packages, homepass, equipment>,
         requestedDecisions=["validate-preconditions", "validate-target-package",
                             "validate-homepass-compatibility", "compute-equipment-delta"]
  outputs: <same shape as VALIDATING_REQUEST>
■
▼ XOR: revalidation passes?
■ NO → REJECT subflow (emit SubscriptionUpgradeRejected
                     with rejectionStage="COMMIT_REVALIDATION")
■ YES ↓
■
[STATE: PENDING_STATE_FLIP]
Service Task: sub-lm-put-pending-status
  inputs: subscriptionId, toStatus="PENDING_UPGRADE",
         currentTransitionType="UPGRADE",
         currentTransitionReasonCode=<upgradeTrigger>
  outputs: priorStatusSnapshot, lmEventId
■
▼
Service Task: sub-lm-compute-cycle-window
  inputs: subscriptionId, referenceDate=NOW
  outputs: cycleWindowJson (cycleStart, cycleEnd, cycleDaysRemaining)
■
▼
[STATE: BILLING_CALL]
Service Task: bil01-emit-intent
  inputs: subscriptionId,
         intentKind="UPGRADE_INTENT",
         intentContext={sourcePackageRef, sourcePackageVersionId, sourcePackagePrice,
                        targetPackageRef, targetPackageVersionId, targetPackagePrice,
                        currency, billingFrequencyChanged, upgradeTrigger,
                        cycleAnchorPolicy=<resolvedCycleAnchorPolicy>},
         cycleWindow
  outputs: intentDecisionId, intentDecision (lineItems with UPGRADE_PRORATE or
         UPGRADE_FIRST_CYCLE + UPGRADE_OLD_CYCLE_CREDIT per policy, totals.net)
■
▼ XOR: pay_first_required AND totals.net > 0?
■ NO ↓ (skip pay-first gate)
■ YES ↓
■   Service Task: bil01-charge-pre-authorize
■   inputs: subscriptionId, chargeCode="UPGRADE_PRORATE",
■           chargeContext={...intentContext...},
■           cycleWindow,
■           chargeAmount=<intentDecision.totals.net>
■   outputs: billableEventId, customerChargeAmount, currency
■   ▼ [STATE: AWAITING_PAYMENT]
■   Message Catch: payment-confirmed (correlated on billableEventId)
■   OR Boundary Timer: 7 days → payment-timeout → REVERT
■   ▼ payment-confirmed
■   Service Task: bil01-confirm-charge
■   inputs: billableEventId
■   outputs: confirmedAt
■
▼
[STATE: FULFILLMENT_CALL]
Service Task: ful-call-modify
  inputs: subscriptionId, homepassId, customerId,
         intent="MODIFY_SERVICE",
         sourcePackageRef, sourcePackageVersionId, sourcePackageCompositionSnapshot,
         targetPackageRef, targetPackageVersionId, targetPackageCompositionSnapshot,
         homePassServiceManagementEndpoints,
         unbindObsoleteEquipment,
         fulfillmentCorrelationId=<operationId>:modify
  outputs: modifyFulfillmentId, modifyTerminalState, perServiceClassResults
■
▼ [STATE: AWAITING_FULFILLMENT_RESPONSE]
Message Catch: fulfillment-completed (correlated on fulfillmentCorrelationId)
OR
Message Catch: fulfillment-failed
■ FAILED → REVERT subflow (bil01-revert-pending-charge if any + sub-lm-revert-to-prior + emit Rejected + finalize FAILED)
■ COMPLETED ↓

```

```

■
▼ XOR: equipmentBindRequired?
■ NO ↓
■ YES ↓
■ Service Task: osr-equipment-bind
■ inputs: subscriptionId, bindings=<from request equipmentBindings>
■ outputs: bindResults
■
▼ XOR: unbindObsoleteEquipment AND equipmentUnbindList non-empty?
■ NO ↓
■ YES ↓
■ Service Task: osr-equipment-unbind
■ inputs: subscriptionId, unbindList=<equipmentUnbindList>,
■ unbindReason="UPGRADE_OBSOLETE"
■ outputs: unbindResults
■
▼
[STATE: COMMITTING_FINAL_STATE]
Service Task: sub-lm-put-master-fields
inputs: subscriptionId,
fieldsJson={
package_ref: <targetPackageRef>,
package_version_id: <targetPackageVersionId>,
previous_package_ref: <sourcePackageRef>,
previous_package_version_id: <sourcePackageVersionId>,
last_status_changed_at: NOW,
// Cycle anchor mutation only when policy = RESET_TO_UPGRADE_DATE
cycle_anchor_day: <if resolvedCycleAnchorPolicy=="RESET_TO_UPGRADE_DATE" then dayOfMonth(NOW) else unchanged>,
cycle_anchor_at: <if resolvedCycleAnchorPolicy=="RESET_TO_UPGRADE_DATE" then NOW else unchanged>
}
outputs: lmEventId
■
▼
Service Task: sub-lm-commit-state
inputs: subscriptionId, toStatus="ACTIVE", expectedFromStatus="PENDING_UPGRADE"
outputs: committedAt
■
▼
[STATE: EMITTING_EVENT]
Service Task: kafka-emit
inputs: topic="sophix.subscription.subscription.upgraded",
key=<subscriptionId>,
payloadJson=<full SubscriptionUpgraded payload per parent $6>
■
▼
Service Task: notification-send (reliability-isolated, conditional on operator config)
inputs: recipientKind="customer", recipientRef=<customerId>,
notificationEventKind="SUBSCRIPTION_UPGRADED",
eventContext={sourcePackageRef, targetPackageRef, currency,
cycleAnchorPolicy=<resolvedCycleAnchorPolicy>,
prorateDelta?, firstCycleCharge?, oldCycleCredit?,
upgradedAt, upgradeTrigger}
■ failure here does NOT fail the BPMN
▼
[STATE: COMPLETED]
Service Task: subscription-operation-finalize
inputs: operationId, finalState="COMPLETED"
■
END (event)

```

2.2 BPMN XML excerpt — conditional pay-first gate + conditional equipment-bind

```

<bpmn:intermediateCatchEvent id="Catch_scheduled_commit_at" name="Wait until scheduledCommitAt">
  <bpmn:timerEventDefinition>
    <bpmn:timeDate>${scheduledCommitAt}</bpmn:timeDate>
  </bpmn:timerEventDefinition>
</bpmn:intermediateCatchEvent>

<bpmn:boundaryEvent id="Boundary_external_cancel" attachedToRef="Catch_scheduled_commit_at" cancelActivity="true">
  <bpmn:messageEventDefinition messageRef="Message_external_cancel_requested" />
</bpmn:boundaryEvent>

<bpmn:sequenceFlow sourceRef="Gateway_immediate_vs_scheduled" targetRef="Catch_scheduled_commit_at">
  <bpmn:conditionExpression>${resolvedEffectiveTiming != "IMMEDIATE"}</bpmn:conditionExpression>
</bpmn:sequenceFlow>

<bpmn:exclusiveGateway id="Gateway_pay_first">
  <bpmn:outgoing>Flow_pay_yes</bpmn:outgoing>
  <bpmn:outgoing>Flow_pay_no</bpmn:outgoing>
</bpmn:exclusiveGateway>
<bpmn:sequenceFlow id="Flow_pay_yes" sourceRef="Gateway_pay_first" targetRef="Task_bil01_pre_auth">
  <bpmn:conditionExpression>${payFirstRequired == true &amp;&amp; intentDecision_totalsNet > 0}</bpmn:conditionExpression>

```

```

</bpmn:sequenceFlow>

<bpmn:serviceTask id="Task_bil01_pre_auth" name="Pre-authorize upgrade-prorate"
    camunda:type="external" camunda:topic="bil01-charge-pre-authorize">
    <bpmn:extensionElements>
        <camunda:inputOutput>
            <camunda:inputParameter name="chargeCode">UPGRADE_PRORATE</camunda:inputParameter>
        </camunda:inputOutput>
    </bpmn:extensionElements>
</bpmn:serviceTask>

<bpmn:intermediateCatchEvent id="Catch_payment_confirmed">
    <bpmn:messageEventDefinition messageRef="Message_payment_confirmed" />
</bpmn:intermediateCatchEvent>
<bpmn:boundaryEvent id="Boundary_payment_timeout" attachedToRef="Catch_payment_confirmed">
    <bpmn:timerEventDefinition><bpmn:timeDuration>P7D</bpmn:timeDuration></bpmn:timerEventDefinition>
</bpmn:boundaryEvent>

<bpmn:exclusiveGateway id="Gateway_equipment_bind">
    <bpmn:outgoing>Flow_bind_yes</bpmn:outgoing>
    <bpmn:outgoing>Flow_bind_no</bpmn:outgoing>
</bpmn:exclusiveGateway>
<bpmn:sequenceFlow id="Flow_bind_yes" sourceRef="Gateway_equipment_bind" targetRef="Task_osr_bind">
    <bpmn:conditionExpression>${equipmentBindRequired == true}</bpmn:conditionExpression>
</bpmn:sequenceFlow>

<bpmn:serviceTask id="Task_osr_bind" name="Bind new equipment"
    camunda:type="external" camunda:topic="osr-equipment-bind">
</bpmn:serviceTask>

```

2.3 Compensation paths

If this step fails	Compensation BPMN runs
drools-decide validation	No mutations yet; emit SubscriptionUpgradeRejected; finalize FAILED
sub-lm-put-pending-status	Atomic; failure leaves subscription ACTIVE with source Package; emit Rejected; finalize FAILED
bil01-emit-intent	sub-lm-revert-to-prior to ACTIVE; emit Rejected; finalize FAILED
bil01-charge-pre-authorize failure	sub-lm-revert-to-prior; emit Rejected; finalize FAILED
payment-timeout (7 days)	bil01-revert-pending-charge; sub-lm-revert-to-prior to ACTIVE with source Package; emit Rejected PAYMENT_TIMEOUT; finalize FAILED
ful-call-modify returns FAILED	bil01-revert-pending-charge (if any); sub-lm-revert-to-prior; emit Rejected; finalize FAILED. FUL-03 handles network-side compensation per its own rules.
osr-equipment-bind failure	FUL-03 MODIFY_SERVICE already applied; manual recovery via manual-recovery-decision user task (network in upgraded state but binding not recorded)
osr-equipment-unbind failure	Logged warning; BPMN proceeds (binding state can be reconciled async; obsolete equipment retained on subscription)
sub-lm-put-master-fields	Manual recovery — package change applied at network level but master fields not updated
sub-lm-commit-state	Manual recovery
kafka-emit	Retried by outbox dispatcher; BPMN proceeds
notification-send	Logged but NOT compensated (reliability-isolated)

2.4 Configuration row (KE WIK)

In subscription_operation_config (framework parent):

operator_code	operation_kind	default_bpmn_process_key	operation_timeout_seconds	feature_flags
WIK	UPGRADE	sub-upgrade-wik	600	{}

(Longer timeout: pay-first gate can wait up to 7 days.)

In subscription_upgrade_config (parent §3.1):

operator_code	customer_self_service_enabled	pay_first_required	default_cycle_anchor_policy	default_effective_timing	max_future_scheduled_days	customer_notification_enabled
WIK	true	true	PRESERVE	IMMEDIATE	90	true

3. Drools package rules.subscription.upgrade.wik

KE WIK rules deployed as part of subscription-workflow-rules KJAR.

3.1 validate-preconditions

```
package rules.subscription.upgrade.wik

import com.sophix.subwf.facts.UpgradeRequestFact
import com.sophix.subwf.facts.SubscriptionStateFact
import com.sophix.subwf.facts.OperatorConfigFact

rule "Eligibility: subscription must be ACTIVE"
when
    $req: UpgradeRequestFact()
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, statusCode != "ACTIVE")
then
    modify($req) { setEligible(false); setEligibilityReasonCode("INVALID_STATE_FOR_UPGRADE") }
end

rule "Eligibility: no in-flight operation"
when
    $req: UpgradeRequestFact(eligible == null)
    $sub: SubscriptionStateFact(subscriptionId == $req.subscriptionId, currentTransitionType != null)
then
    modify($req) { setEligible(false); setEligibilityReasonCode("OPERATION_IN_PROGRESS") }
end

rule "Eligibility: CUSTOMER_REQUESTED_UPGRADE requires CUSTOMER_SELF_SERVICE / CALL_CENTER / SUBSCRIPTION_OPERATOR role"
when
    $req: UpgradeRequestFact(upgradeTrigger == "CUSTOMER_REQUESTED_UPGRADE", eligible == null,
        initiatingActorRole not in ("CUSTOMER_SELF_SERVICE", "CALL_CENTER", "SUBSCRIPTION_OPERATOR"))
then
    modify($req) { setEligible(false); setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER") }
end

rule "Eligibility: ADMIN_UPGRADE requires SUBSCRIPTION_ADMIN or SUBSCRIPTION_OPERATOR"
when
    $req: UpgradeRequestFact(upgradeTrigger == "ADMIN_UPGRADE", eligible == null,
        initiatingActorRole not in ("SUBSCRIPTION_ADMIN", "SUBSCRIPTION_OPERATOR"))
then
    modify($req) { setEligible(false); setEligibilityReasonCode("ROLE_NOT_AUTHORIZED_FOR_TRIGGER") }
end

rule "Eligibility: CUSTOMER_SELF_SERVICE requires operator config enabled"
when
    $req: UpgradeRequestFact(initiatingActorRole == "CUSTOMER_SELF_SERVICE", eligible == null)
    $cfg: OperatorConfigFact(customerSelfServiceEnabled == false)
then
    modify($req) { setEligible(false); setEligibilityReasonCode("CUSTOMER_SELF_SERVICE_NOT_ENABLED") }
end

rule "Eligibility: default-pass"
salience -100
when
    $req: UpgradeRequestFact(eligible == null)
then
    modify($req) { setEligible(true) }
end
```

3.2 validate-target-package

```
import com.sophix.subwf.facts.PackageFact

rule "Target: same as source – blocked"
when
    $req: UpgradeRequestFact(eligible == true, targetValid == null)
    eval($req.getTargetPackageRef().equals($req.getSourcePackageRef()))
then
    modify($req) { setTargetValid(false); setTargetReasonCode("SAME_PACKAGE") }
end

rule "Target: status must be ACTIVE"
when
    $req: UpgradeRequestFact(eligible == true, targetValid == null)
    $tgt: PackageFact(packageRef == $req.targetPackageRef, status != "ACTIVE")
then
    modify($req) { setTargetValid(false); setTargetReasonCode("TARGET_PACKAGE_NOT_ACTIVE") }
end
```

```

rule "Target: currency must match subscription currency"
when
    $req: UpgradeRequestFact(eligible == true, targetValid == null)
    $tgt: PackageFact(packageRef == $req.targetPackageRef)
    eval(!$tgt.getCurrency().equals($req.getCurrentCurrency()))
then
    modify($req) { setTargetValid(false); setTargetReasonCode("CURRENCY_MISMATCH") }
end

rule "Target: price must be ≥ source price"
when
    $req: UpgradeRequestFact(eligible == true, targetValid == null)
    $tgt: PackageFact(packageRef == $req.targetPackageRef)
    eval($tgt.getPrice().compareTo($req.getSourcePackagePrice()) < 0)
then
    modify($req) { setTargetValid(false); setTargetReasonCode("NOT_AN_UPGRADE") }
end

rule "Target: default-pass"
salience -100
when
    $req: UpgradeRequestFact(eligible == true, targetValid == null)
then
    modify($req) { setTargetValid(true) }
end

```

3.3 validate-homepass-compatibility

```

import com.sophix.subwf.facts.HomePassFact
import com.sophix.subwf.facts.HomePassStatusCodeFact

rule "HomePass: status_code must be active"
when
    $req: UpgradeRequestFact(eligible == true, targetValid == true, homePassCompatible == null)
    $hp: HomePassFact(homePassId == $req.homePassId)
    $sc: HomePassStatusCodeFact(code == $hp.statusCode, isActive == false)
then
    modify($req) {
        setHomePassCompatible(false);
        setHomePassReasonCode("HOMEPASS_NOT_ACTIVE")
    }
end

rule "HomePass: missing service_management_endpoints for required service family"
when
    $req: UpgradeRequestFact(eligible == true, targetValid == true, homePassCompatible == null)
    $hp: HomePassFact(homePassId == $req.homePassId)
    $tgt: PackageFact(packageRef == $req.targetPackageRef)
    eval(!$hp.getServiceManagementEndpoints().keySet().containsAll($tgt.getRequiredServiceManagementKeys()))
then
    java.util.Set missing = new java.util.HashSet($tgt.getRequiredServiceManagementKeys());
    missing.removeAll($hp.getServiceManagementEndpoints().keySet());
    modify($req) {
        setHomePassCompatible(false);
        setHomePassReasonCode("PACKAGE_NOT_AVAILABLE_AT_HOMEPASS");
        setMissingServiceManagementKeys(new java.util.ArrayList(missing))
    }
end

rule "HomePass: port not provisioned for required service family (AT_INSTALLATION)"
when
    $req: UpgradeRequestFact(eligible == true, targetValid == true, homePassCompatible == null)
    $hp: HomePassFact(homePassId == $req.homePassId)
    $tgt: PackageFact(packageRef == $req.targetPackageRef)
then
    java.util.List unprovisioned = new java.util.ArrayList();
    for (Object key : $tgt.getRequiredServiceManagementKeys()) {
        Object endpoint = $hp.getServiceManagementEndpoints().get(key);
        if (endpoint != null && ((java.util.Map) endpoint).get("port") == null) {
            unprovisioned.add(key);
        }
    }
    if (!unprovisioned.isEmpty()) {
        modify($req) {
            setHomePassCompatible(false);
            setHomePassReasonCode("HOMEPASS_PORT_NOT_PROVISIONED");
            setUnprovisionedPorts(unprovisioned)
        }
    }
end

rule "HomePass: default-pass"

```

```

salience -100
when
  $req: UpgradeRequestFact(homePassCompatible == null, eligible == true, targetValid == true)
then
  modify($req) {
    setHomePassCompatible(true);
    setMissingServiceManagementKeys(java.util.Collections.emptyList());
    setUnprovisionedPorts(java.util.Collections.emptyList())
  }
end

```

3.4 compute-equipment-delta

```

import com.sophix.subwf.facts.ServiceClassFact
import com.sophix.subwf.facts.OsrBindingFact

rule "Equipment delta: new service classes requiring equipment + no existing binding → bind required"
when
  $req: UpgradeRequestFact(homePassCompatible == true, equipmentBindRequired == null)
  $tgt: PackageFact(packageRef == $req.targetPackageRef)
  $src: PackageFact(packageRef == $req.sourcePackageRef)
then
  java.util.Set newClasses = new java.util.HashSet($tgt.getRequiredServiceClasses());
  newClasses.removeAll($src.getRequiredServiceClasses());

  java.util.List unfulfilled = new java.util.ArrayList();
  for (Object scId : newClasses) {
    ServiceClassFact sc = lookupServiceClass(scId);
    if (sc != null && sc.isRequiresEquipment()) {
      boolean covered = $req.getEquipmentBindings().stream()
        .anyMatch(b -> b.getFulfillsServiceClass().equals(scId));
      if (!covered) {
        unfulfilled.add(scId);
      }
    }
  }

  modify($req) {
    setEquipmentBindRequired(!unfulfilled.isEmpty() || !$req.getEquipmentBindings().isEmpty());
    setUnfulfilledServiceClasses(unfulfilled)
  }
end

rule "Equipment delta: unbind list if unbindObsoleteEquipment + service class no longer required"
when
  $req: UpgradeRequestFact(unbindObsoleteEquipment == true, equipmentUnbindList == null)
  $src: PackageFact(packageRef == $req.sourcePackageRef)
  $tgt: PackageFact(packageRef == $req.targetPackageRef)
then
  java.util.Set obsoleteClasses = new java.util.HashSet($src.getRequiredServiceClasses());
  obsoleteClasses.removeAll($tgt.getRequiredServiceClasses());

  java.util.List unbindList = $req.getCurrentBindings().stream()
    .filter(b -> obsoleteClasses.contains(b.getServiceClassId()))
    .map(b -> b.getEquipmentId())
    .collect(java.util.stream.Collectors.toList());

  modify($req) { setEquipmentUnbindList(unbindList) }
end

rule "Equipment delta: default (no bind required)"
salience -100
when
  $req: UpgradeRequestFact(equipmentBindRequired == null)
then
  modify($req) {
    setEquipmentBindRequired(false);
    setUnfulfilledServiceClasses(java.util.Collections.emptyList());
    setEquipmentUnbindList(java.util.Collections.emptyList())
  }
end

```

3.5 resolve-cycle-anchor-policy

```

rule "Cycle anchor: request supplied PRESERVE"
when
  $req: UpgradeRequestFact(cycleAnchorPolicy == "PRESERVE", policyValid == null)
then
  modify($req) {
    setResolvedCycleAnchorPolicy("PRESERVE");
    setPolicyValid(true)
  }
end

```

```

    }
end

rule "Cycle anchor: request supplied RESET_TO_UPGRADE_DATE"
when
    $req: UpgradeRequestFact(cycleAnchorPolicy == "RESET_TO_UPGRADE_DATE", policyValid == null)
then
    modify($req) {
        setResolvedCycleAnchorPolicy("RESET_TO_UPGRADE_DATE");
        setPolicyValid(true)
    }
end

rule "Cycle anchor: request supplied unknown value – reject"
when
    $req: UpgradeRequestFact(cycleAnchorPolicy != null, policyValid == null,
        cycleAnchorPolicy not in ("PRESERVE", "RESET_TO_UPGRADE_DATE"))
then
    modify($req) {
        setPolicyValid(false);
        setPolicyReasonCode("UNKNOWN_CYCLE_ANCHOR_POLICY")
    }
end

rule "Cycle anchor: no value supplied – default from operator config"
when
    $req: UpgradeRequestFact(cycleAnchorPolicy == null, policyValid == null)
    $cfg: OperatorConfigFact()
then
    modify($req) {
        setResolvedCycleAnchorPolicy($cfg.getDefaultCycleAnchorPolicy());
        setPolicyValid(true)
    }
end

```

3.6 resolve-effective-timing

```

import com.sophix.subwf.facts.CycleWindowFact
import java.time.Instant
import java.time.Duration

rule "Effective timing: no value supplied – default from operator config"
when
    $req: UpgradeRequestFact(effectiveTiming == null, timingValid == null)
    $cfg: OperatorConfigFact()
then
    modify($req) {
        setEffectiveTiming($cfg.getDefaultEffectiveTiming())
    }
end

rule "Effective timing: unknown value – reject"
when
    $req: UpgradeRequestFact(timingValid == null,
        effectiveTiming not in ("IMMEDIATE", "END_OF_CURRENT_CYCLE", "SCHEDULED_AT"))
then
    modify($req) {
        setTimingValid(false);
        setTimingReasonCode("UNKNOWN_EFFECTIVE_TIMING")
    }
end

rule "Effective timing: SCHEDULED_AT requires scheduledAt"
when
    $req: UpgradeRequestFact(effectiveTiming == "SCHEDULED_AT", scheduledAt == null, timingValid == null)
then
    modify($req) {
        setTimingValid(false);
        setTimingReasonCode("SCHEDULED_AT_MISSING")
    }
end

rule "Effective timing: SCHEDULED_AT out of bounds"
when
    $req: UpgradeRequestFact(effectiveTiming == "SCHEDULED_AT", scheduledAt != null, timingValid == null)
    $cfg: OperatorConfigFact()
    eval(
        Instant.parse($req.getScheduledAt()).isBefore(Instant.now())
        || Duration.between(Instant.now(), Instant.parse($req.getScheduledAt())).toDays() > $cfg.getMaxFutureScheduledDays()
    )
then
    modify($req) {

```

```

        setTimingValid(false);
        setTimingReasonCode("SCHEDULED_AT_OUT_OF_BOUNDS")
    }
end

rule "Effective timing: END_OF_CURRENT_CYCLE + RESET is redundant"
when
    $req: UpgradeRequestFact(effectiveTiming == "END_OF_CURRENT_CYCLE",
                             resolvedCycleAnchorPolicy == "RESET_TO_UPGRADE_DATE",
                             timingValid == null)
then
    modify($req) {
        setTimingValid(false);
        setTimingReasonCode("REDUNDANT_POLICY_COMBINATION")
    }
end

rule "Effective timing: IMMEDIATE resolved"
when
    $req: UpgradeRequestFact(effectiveTiming == "IMMEDIATE", timingValid == null)
then
    modify($req) {
        setResolvedEffectiveTiming("IMMEDIATE");
        setScheduledCommitAt(null);
        setTimingValid(true)
    }
end

rule "Effective timing: END_OF_CURRENT_CYCLE resolved"
when
    $req: UpgradeRequestFact(effectiveTiming == "END_OF_CURRENT_CYCLE", timingValid == null)
    $cyc: CycleWindowFact(subscriptionId == $req.subscriptionId)
then
    modify($req) {
        setResolvedEffectiveTiming("END_OF_CURRENT_CYCLE");
        setScheduledCommitAt($cyc.getCycleEnd());
        setTimingValid(true)
    }
end

rule "Effective timing: SCHEDULED_AT resolved"
when
    $req: UpgradeRequestFact(effectiveTiming == "SCHEDULED_AT", scheduledAt != null, timingValid == null)
then
    modify($req) {
        setResolvedEffectiveTiming("SCHEDULED_AT");
        setScheduledCommitAt($req.getScheduledAt());
        setTimingValid(true)
    }
end

```

4. Worked example — Kamau upgrades from 100Mbps to 250Mbps

Context: Kamau is on pkg_INET_100M_VOIP_RES (KES 2,500/month). He calls KE Call Center on 2026-05-20 to upgrade to pkg_INET_250M_VOIP_RES (KES 4,500/month). His cycle anchor is the 1st of the month; cycle 2026-05-01 to 2026-05-31; 11 days remaining (May 21-31). Both packages compose the same Service classes (INET + VOIP), so no equipment change needed.

14:00:00 — Trigger endpoint:

```

POST /api/subscriptions/sub_01HZ_KAMAU_FIBER/upgrade
Idempotency-Key: upgrade-sub_01HZ_KAMAU-pkg_INET_250-20260520

{ "operatorCode": "WIK", "upgradeTrigger": "CUSTOMER_REQUESTED_UPGRADE",
  "targetPackageRef": "pkg_01HX_INET_250M_VOIP_RES",
  "cycleAnchorPolicy": "PRESERVE",
  "equipmentBindings": [], "unbindObsoleteEquipment": false,
  "notes": "Customer requested speed upgrade after promotion notice",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole": "CALL_CENTER",
  "initiatingWorkflowKind": "BO_UI",
  "initiatingWorkflowRef": "cc-ticket-2026-05-20-4421" }
→ 202 operationId=subop_01HZ_KAMAU_UPGRADE, currentState=VALIDATING,
   resolvedCycleAnchorPolicy="PRESERVE"

```

Camunda starts pi_01HZ_KAMAU_UPGRADE of sub-upgrade-wik.

14:00:00.5 — VALIDATING: drools-decide invokes rules.subscription.upgrade.wik with all 5 decisions. Facts: subscription ACTIVE, no in-flight ops, role CALL_CENTER on CUSTOMER_REQUESTED_UPGRADE ✓, target Package pkg_01HX_INET_250M_VOIP_RES ACTIVE, currency KES ✓, price KES 4,500 ≥ KES 2,500 ✓, target ≠ source ✓. HomePass hp_01HZ_KAREN_017 has service_management_endpoints = {data: {nodeCode: "OLT-NRB-WTL-01", port: "0/1/12"}, voice: {nodeCode: "VOIPSWITCH-NRB-01", port: "5012"}}, status ACT (is_active=true). Target Package requires service-management keys {data, voice} — both present + ports non-null ✓. Equipment delta: target requires same service classes as source (INET + VOIP), existing bindings cover both, no new bindings required, equipmentBindRequired=false. Cycle-anchor policy: request supplied PRESERVE, resolved as-is, policyValid=true. Decisions: all ✓.

14:00:01 — PENDING_STATE_FLIP: sub-lm-put-pending-status flips subscription to PENDING_UPGRADE; sets current_transition_type=UPGRADE, current_transition_reason_code=CUSTOMER_REQUESTED_UPGRADE.

14:00:01.5: sub-lm-compute-cycle-window returns {cycleStart:"2026-05-01", cycleEnd:"2026-05-31", cycleDaysRemaining:11} (May 21 onward).

14:00:02 — BILLING_CALL: bil01-emit-intent calls BIL-01 with intentKind=UPGRADE_INTENT, context {sourcePackagePrice:2500, targetPackagePrice:4500, currency:"KES", cycleDaysRemaining:11, cycleAnchorPolicy:"PRESERVE", ...}. BIL-01 sees PRESERVE, computes proration: $(4500 - 2500) \times 11 / 31 = 709.68$ KES. (Actual rounding per BIL-01 policy; assume 710.00 KES.) Returns lineItems=[{code:"UPGRADE_PRORATE", amount:710.00, currency:"KES"}], totals.net=710.00.

14:00:03: Pay-first gateway: pay_first_required=true AND totals.net=710 > 0 → run pay-first gate.

14:00:03 — bil01-charge-pre-authorize: BIL-01 creates BillableEvent bil_evt_01HZ_KAMAU_UPGRADE_PRORATE with status PENDING, amount 710.00 KES. Returns billableEventId, customerChargeAmount=710.00.

14:00:03 — AWAITING_PAYMENT: BPMN message catch armed for payment-confirmed correlated on billableEventId. CC agent processes Kamau's M-Pesa STK push; payment confirmed at 14:01:30.

14:01:30: Message payment-confirmed fires. bil01-confirm-charge confirms BillableEvent → status SETTLED.

14:01:31 — FULFILLMENT_CALL: ful-call-modify calls FUL-03 with intent=MODIFY_SERVICE, source + target Package snapshots, HomePass service_management_endpoints snapshot, fulfillmentCorrelationId=subop_01HZ_KAMAU_UPGRADE:modify. FUL-03 diffs the compositions: {data, voice} in both source and target → call modifyService on each. FUL-03 looks up service_management_endpoints['data'] → {OLT-NRB-WTL-01, 0/1/12}; dispatches to GponResProvisioner.modifyService (push 250Mbps profile to OLT for Kamau's port). Looks up service_management_endpoints['voice'] → {VOIPSWITCH-NRB-01, 5012}; dispatches to VoipSipProvisioner.modifyService (no profile change needed — voice unchanged across the upgrade). No new service classes to activate, no obsolete to deactivate. Returns terminalState=FULLY_FULFILLED after 1.8 seconds.

14:01:32.8 — AWAITING_FULFILLMENT_RESPONSE: Camunda message catch fulfillment-completed fires.

14:01:33: Equipment-bind gateway: equipmentBindRequired=false → skip. Equipment-unbind gateway: unbindObsoleteEquipment=false → skip.

14:01:33 — COMMITTING_FINAL_STATE: sub-lm-put-master-fields writes package_ref=pkg_01HX_INET_250M_VOIP_RES, package_version_id=pkv_01HX_INET_250M_V1, previous_package_ref=pkg_01HX_INET_100M_VOIP_RES, previous_package_version_id=pkv_01HX_INET_100M_V3, last_status_changed_at=NOW. cycle_anchor_day and cycle_anchor_at are NOT mutated (policy = PRESERVE).

sub-lm-commit-state flips PENDING_UPGRADE → ACTIVE.

14:01:34 — EMITTING_EVENT: kafka-emit writes full SubscriptionUpgraded payload to outbox.

14:01:34.5: notification-send invokes NOT-01 with notificationEventKind=SUBSCRIPTION_UPGRADED and event context {sourcePackageRef, targetPackageRef, proratedDelta:710.00, currency:"KES", upgradedAt}. NOT-01 resolves Kamau's preferred channel + template + language; dispatches the upgrade confirmation.

14:01:35 — COMPLETED: subscription-operation-finalize sets final_state=COMPLETED. Total operation duration: ~95 seconds (mostly waiting for customer M-Pesa payment confirmation).

14:01:35+ (downstream consumers):

- BIL-02 consumes SubscriptionUpgraded; future cycles will accrue at KES 4,500/month from 2026-06-01.
- CVM analytics records the upgrade event for ARPU tracking.
- EM-04 reporting indexes the upgrade for monthly KPI metrics.

- SIP-01 audit captures the version-pinning shift (kpv_01HX_INET_100M_V3 → pkv_01HX_INET_250M_V1).

Kamau's subscription is now on the 250Mbps package. His OLT bandwidth profile is updated. The prorated KES 710 for the 11 remaining May days has been charged. June 1 onward, full KES 4,500/month applies.

Alternative path — cycleAnchorPolicy = RESET_TO_UPGRADE_DATE

If Kamau had instead requested cycleAnchorPolicy=RESET_TO_UPGRADE_DATE, BIL-01's UPGRADE_INTENT decision would have returned two line items: UPGRADE_FIRST_CYCLE for KES 4,500 (fresh cycle May 20 → June 19 at target price), and UPGRADE_OLD_CYCLE_CREDIT for the unused portion of the May 1-31 cycle at the source price ($\text{KES } 2,500 \times 11/31 \approx \text{KES } 887.10$, credited per BIL-01 policy). totals.net would have been $4500.00 - 887.10 = 3612.90$ KES. The master-fields step would have updated cycle_anchor_day=20, cycle_anchor_at=2026-05-20T14:01:33+03:00. Subsequent cycles would roll forward from 20th of month.

Alternative path — double-play → triple-play (service-class expansion + new equipment)

Consider Otieno: on pkg_INET_VOIP_RES (double-play; INET + VOIP), upgrading to pkg_INET_VOIP_IPTV_RES (triple-play; INET + VOIP + IPTV). His HomePass hp_01HZ_LAVI_042 was deployed behind an OLT that is configured for IPTV multicast — so service_management_endpoints = {data: {OLT-NRB-LAV-01, 0/2/03}, voice: {VOIPSWITCH-NRB-01, 5103}, iptv_multicast: {OLT-NRB-LAV-01, 0/2/03}}. His current equipment bindings: ONT (fulfills data + iptv_multicast), ATA (fulfills voice). Triple-play requires an STB to fulfill the IPTV service class on the customer side.

Trigger endpoint:

```
POST /api/subscriptions/sub_01HZ_OTIENO_FIBER/upgrade
{
  "operatorCode": "WIK",
  "upgradeTrigger": "CUSTOMER_REQUESTED_UPGRADE",
  "targetPackageRef": "pkg_01HX_INET_VOIP_IPTV_RES",
  "cycleAnchorPolicy": "PRESERVE",
  "equipmentBindings": [
    { "equipmentId": "eqp_STB_01HZ_LAVI_042", "serviceClassId": "IPTV", "fulfillsServiceClass": "IPTV" }
  ],
  "unbindObsoleteEquipment": false,
  ...
}
```

VALIDATING: Drools facts: subscription ACTIVE, target Package ACTIVE/KES/price ≥ source ✓, HomePass status ACT ✓, HomePass service_management_endpoints keys include {data, voice, iptv_multicast} — covers target's required keys ✓, all ports non-null ✓. Equipment delta: target requires service class IPTV not in source; PLM-CFG-01 says IPTV.requires_equipment=true; STB binding supplied in request fulfills it → equipmentBindRequired=true, unfulfilledServiceClasses=[]. Decisions all ✓.

BILLING_CALL: bil01-emit-intent with UPGRADE_INTENT. BIL-01 computes proration $\text{delta} = (\text{target_price} - \text{source_price}) \times \text{cycle_days_remaining} / \text{cycle_days_total}$. Returns UPGRADE_PRORATE line item.

FULFILLMENT_CALL: ful-call-modify calls FUL-03 with source + target Package snapshots + HomePass service_management_endpoints. FUL-03 diffs compositions:

- data (both) → GponResProvisioner.modifyService at {OLT-NRB-LAV-01, 0/2/03} (no bandwidth change in this example; could be a no-op or rate-tier touch).
- voice (both) → VoipSipProvisioner.modifyService at {VOIPSWITCH-NRB-01, 5103} (no-op).
- iptv_multicast (target only, NEW) → GponResProvisioner.activate at {OLT-NRB-LAV-01, 0/2/03} (provision IGMP-snooping for Otieno's ONT port, join the multicast group, register STB serial in the IPTV middleware's AAA channel package).

Each provisioner uses the resolved node + port from service_management_endpoints. Returns terminalState=FULLY_FULFILLED.

OSR-equipment-bind: Binds the STB to Otieno's subscription against service class IPTV. Records eqp_STB_01HZ_LAVI_042 → sub_01HZ_OTIENO_FIBER.

COMMITTING_FINAL_STATE: master fields updated: package_ref + package_version_id + previous_package_ref + previous_package_version_id. HomePass row UNCHANGED (RLM-CFG-01 saw no mutation request — physical capability iptv_multicast already existed in service_management_endpoints before the upgrade).

Result: Otieno is now on triple-play. The HomePass capability was already present (physical multicast-capable OLT path); what changed was per-customer provisioning at that capability + the STB binding on the subscription side.

Alternative path — effectiveTiming = END_OF_CURRENT_CYCLE

Customer Wanjiku is on pkg_INET_100M_VOIP_RES (KES 2,500/month) with cycle anchor on the 1st. On May 20 she calls CC: "I'd like to upgrade to the 250Mbps plan starting next month — no extra charge for this month." CC submits with effectiveTiming=END_OF_CURRENT_CYCLE, cycleAnchorPolicy=PRESERVE.

May 20, 14:00 — Trigger endpoint accepted. subscription_operation row written with current_state=SCHEDULED. BPMN starts at VALIDATING_REQUEST. Drools resolves all decisions: eligible=true, targetValid=true, homePassCompatible=true, equipmentBindRequired=false, resolvedCycleAnchorPolicy=PRESERVE, resolvedEffectiveTiming=END_OF_CURRENT_CYCLE, scheduledCommitAt=2026-05-31T23:59:59+03:00. Workflow emits SubscriptionUpgradeScheduled on Kafka. NOT-01 notifies Wanjiku "Your upgrade to 250Mbps is scheduled for June 1." BPMN parks on Camunda intermediate timer with timeDate=\${scheduledCommitAt}. Subscription stays ACTIVE on the source 100Mbps Package; May cycle continues to bill at KES 2,500.

May 31, 23:59:59 — Timer fires. BPMN advances to VALIDATING. Drools re-runs the 4 validation decisions (preconditions, target package, homepass compatibility, equipment delta). All pass. State flips PENDING_UPGRADE. sub-lm-compute-cycle-window returns the new window starting June 1. bil01-emit-intent calls BIL-01 with UPGRADE_INTENT, context including cycleAnchorPolicy=PRESERVE, effectiveTiming=END_OF_CURRENT_CYCLE. BIL-01 sees commit-at-cycle-boundary case: zero proration delta, just sets next cycle to target price. Returns lineItems=[], totals.net=0.

Pay-first gate skipped (totals.net = 0). ful-call-modify → FUL-03 dispatches MODIFY_SERVICE to GponResProvisioner (push 250Mbps profile). sub-lm-put-master-fields updates package_ref + previous_package_ref. sub-lm-commit-state flips to ACTIVE. kafka-emit publishes SubscriptionUpgraded with resolvedEffectiveTiming=END_OF_CURRENT_CYCLE, scheduledCommitAt=2026-05-31T23:59:59+03:00, prorateDelta=null (no delta — cycle boundary commit).

June 1 onward: Wanjiku billed at KES 4,500/month, cycle anchor still on the 1st. Total customer-visible behavior: she chose her upgrade on May 20, paid nothing extra for May, gets the new plan automatically on June 1.

Cancellation flow

If on May 22 Wanjiku had called back to cancel: DELETE /subscriptions/sub_01HZ_WANJIKU/pending-upgrade/subop_01HZ_WANJIKU_UPGRADE_EOC with cancellationReasonCode=CUSTOMER_CHANGED_MIND. The cancel REST endpoint sends a external_cancel_requested message correlated on the operation id; Camunda's boundary event on the timer fires; the BPMN exits the wait, emits SubscriptionUpgradeCancelled, finalizes the operation as CANCELLED. NOT-01 confirms to Wanjiku. Subscription stays on source Package.

5. Cross-DD coordination

Module	Direction	Contract
SUB-WF-UPGRADE-01 (parent)	This satellite implements parent's contract	Triggers, target-package validation, equipment-delta rules all defined in parent
SUB-WF-FRAMEWORK-01 (framework parent)	This satellite composes framework primitives	subscription_operation row written; BPMN started
SUB-WF-FRAMEWORK-01-WORKERS	This satellite references workers by topic	13 workers composed
SUB-LM-01	Workers write status + master fields	Status flip + package refs
SIP-01	Workers read Package + Package Version catalog	Target Package validation
PLM-CFG-01	Workers read service_class definitions	Equipment-requirement delta
RLM-CFG-01	Workers read HomePass service_management_endpoints + status_code	HomePass physical-capability check; read-only
BIL-01	bil01-emit-intent + pay-first gate	UPGRADE_INTENT with proration
BIL-CFG-01	BillableEvent catalog	Upgrade-prorate definitions
FUL-03	ful-call-modify worker	MODIFY_SERVICE. Owns per-service-class fan-out via HomePass service_management_endpoints
OSR-INSTANCE-01	osr-equipment-bind / unbind workers	Equipment binding mutations
NOT-01	notification-send worker	Customer notification per operator config
BIL-02 / CVM / EM-04	Consume Kafka events	New-price cycle accrual, analytics, reporting

5.1 Stub debt + open coordination items

- WUG / WTZ / YASSN FLOW satellites — pending operator deep dives.

B — Current TZ

Status: To be filled in after codebase cartography review.

Legacy package upgrade flow exists; specific implementation shape — proration calculation, equipment binding propagation, network reconfiguration sequencing — requires source-code review to document. Until that cartography lands, this satellite does not assert specific legacy structural claims.